

JEGYZŐKÖNYV

Webes adatkezelő környezetek

Féléves feladat

Sportverseny nyilvántartás

Készítette: **Kovács Attila Marcell**

Neptunkód: **SEGUV3**

Dátum: 2025. **december**

Miskolc, 2025

Tartalomjegyzék

1. XML adatkezelő rendszer	5
1.1 Az adatbázis ER modell tervezése	5
1.2 Az adatbázis konvertálása XDM modellre.....	7
1.3 Az XDM modell alapján XML dokumentum készítése.....	8
1.4 Az XML dokumentum alapján XMLSchema készítése.....	10
2. DOM program	13
2.1 Adatolvasás	13
2.2 Adat-lekérdezés.....	15
2.3 Adatmódosítás.....	18

Bevezetés

A fél éves beadandó feladat célja egy olyan sportverseny-nyilvántartó rendszer elkészítése volt, amely egységes és áttekinthető formában kezeli a sportolók, csapatok, versenyek, helyszínek, szövetségek és részvételek adatait.

A projekt középpontjában az XML technológia állt, mivel ez lehetőséget biztosít az adatok hierarchikus, strukturált és platformfüggetlen tárolására, valamint könnyen feldolgozható formát ad különböző programozási nyelvek és rendszerek számára.

Első lépésként elkészült az ER-modell, amely meghatározta az entitásokat és a közöttük lévő kapcsolatokat. Ezt követően az ER-modellt XDM-modellé alakítottam, amely már az XML-dokumentum szerkezetéhez igazodó logikai felépítést mutatja.

Az XDM-modell alapján létrehoztam a validált SEGUV3_XML.xml dokumentumot, valamint a hozzá tartozó SEGUV3_XMLSchema.xsd sémát, amely biztosítja a dokumentum érvényességét és hibamentességét.

A munka második részében egy DOM (Document Object Model) alapú Java alkalmazást készítettem, amely képes az XML-fájl beolvasására, lekérdezésére és adatmódosítására.

A végeredmény egy működő, példaszerű sportverseny-adatkezelő rendszer, amely technikailag bemutatja az XML és a DOM feldolgozás lehetőségeit, és gyakorlati példán keresztül szemlélteti ezek alkalmazását egy valós problémakörben.

A feladat leírása

A rendszer célja egy sportverseny-nyilvántartás megvalósítása, amelyben a sportolók, csapatok, versenyek, helyszínek és részvételi adatok strukturált módon kerülnek tárolásra és egymással összekapcsolásra.

A cél egy olyan egységes adatkezelő megoldás létrehozása volt, amely a sportesemények adminisztrációját, az eredmények nyilvántartását és az adatok visszakereshetőségét is támogatja. A megoldás az XML technológiára épül, ezért az adatok hierarchikus, jól strukturált formában jelennek meg, ami átláthatóvá és könnyen kezelhetővé teszi a dokumentumot.

Az adatmodell hat fő entitást tartalmaz:

- **Sportoló** – a versenyzők személyes adatai és elért eredményei,
- **Csapat** – az adott sportolók által alkotott egységek és azok jellemzői,
- **Verseny** – az események megnevezése, időpontja és kategóriája,
- **Helyszín** – a versenyek lebonyolításának helye, ország, város és létesítmény szerint,
- **Szövetség** – a sportági szövetségek adatait tartalmazza, mint például a név, sportág, alapítás éve és központ, valamint az általuk felügyelt csapatok,
- **Részvétel** – a csapatok és versenyek kapcsolatát, valamint az elért helyezéseket és díjazásokat leíró adatok.

A dokumentum érvényességét az XSD séma biztosítja, amely részletesen meghatározza az adatok típusait, a kötelező és opcionális elemeket, valamint a kapcsolatok logikai szerkezetét. Ez a validációs mechanizmus garantálja, hogy az XML fájl csak szabályos, ellenőrzött adatokat tartalmazzon, így kizárja a szerkezeti hibákat vagy hibás formátumokat.

A programozási részben a DOM API (Document Object Model) segítségével valósult meg az adatok kezelése. A rendszer képes az XML dokumentum beolvasására, a benne tárolt elemek lekérdezésére, új adatok hozzáadására, valamint a már meglévő adatok módosítására és mentésére. Ezáltal a program nemcsak statikus adatokat tárol, hanem egy dinamikusan kezelhető, interaktív XML alapú adatnyilvántartást valósít meg.

A megvalósítás célja egy olyan valósághűen modellezett, oktatási célú sportverseny adatkezelő rendszer létrehozása, amely bemutatja az XML és DOM technológiák gyakorlati alkalmazását, valamint az adatmodellezés és adatfeldolgozás közötti kapcsolatot. A feladat elvégzése hozzájárul az XML alapú rendszerek működésének megértéséhez, az adatstruktúrák helyes megtervezéséhez és a valós alkalmazásokban történő felhasználásához.

1. XML adatkezelő rendszer

1.1 Az adatbázis ER modell tervezése

A rendszer tervezésének első lépéseként elkészült a sportverseny-nyilvántartás ER (Entity–Relationship) modellje, amely az adatok közötti logikai kapcsolatok és szerkezeti összefüggések szemléltetésére szolgál. A modell célja, hogy a sportversenyekhez kapcsolódó információkat, mint például a sportolók, csapatok, versenyek, helyszínek, részvételi adatok és szövetségek egységes, átlátható és logikailag konzisztens formában ábrázolja.

Az ER modell megalkotása lehetőséget biztosít az adatbázis-struktúra logikai szintű elemzésére, még azelőtt, hogy az XML-re és XSD-re épülő fizikai modell létrejönne.

A modellben hat fő egyed található, amelyek a rendszer működésének központi szereplőit és azok jellemzőit írják le:

- **Sportoló** – a versenyzők személyes és teljesítményadatait tartalmazza, mint a név (összetett attribútum: vezetéknév és keresztnév), nemzetiség, a születési_dátum, valamint az elért_eredmények, amely többértékű attribútumként jelenik meg, mivel egy sportoló több versenyen is eredményt érhet el.
- **Csapat** – a sportolókhöz tartozó szervezeti egységet jelöli, amelynek attribútumai a csapat neve, sportága, országa, valamint az edző_nev, amely szintén összetett (vezetéknév + keresztnév).
- **Verseny** – a sportesemények adatait tartalmazza, többek között a verseny nevét, dátumát, kategóriáját és szintjét.
- **Helyszín** – a versenyek lebonyolításának fizikai helyét írja le (város, ország, létesítmény_nev és befogadó_képesség).
- **Részvétel** – a Csapat és a Verseny közötti kapcsolatot valósítja meg. E kapcsolatnak önálló tulajdonságai is vannak, például helyezés, pontszám, díj_összeg és megjegyzés, amelyek a versenyek eredményeinek pontos rögzítését szolgálják.
- **Szövetség** – a csapatokat összefogó sportági szervezeteket reprezentálja. Attribútumai: sz_id (azonosító), név, sportág, alapítási_év és központ. Egy szövetséghez több csapat is tartozhat, ugyanakkor egy csapat jellemzően csak egy sportági szövetséghez kapcsolódik.

Kapcsolatok a modellben

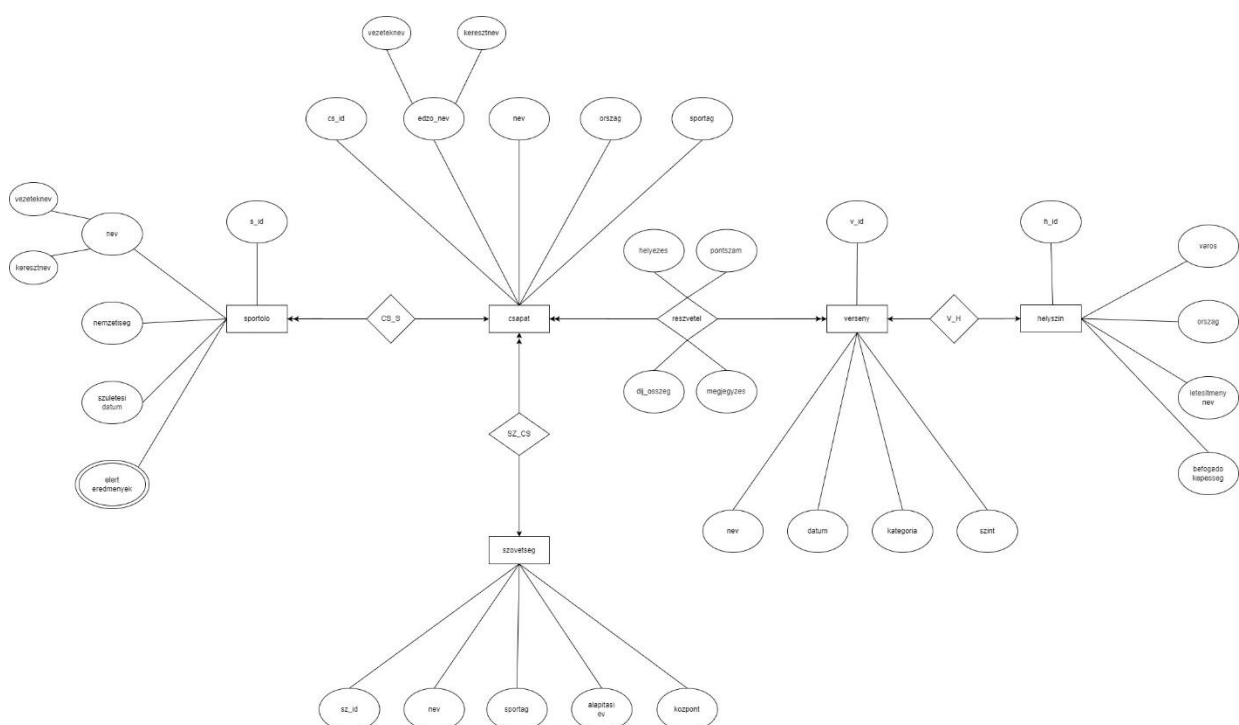
- **1:N kapcsolat** a Sportoló és a Csapat között, mivel egy csapat több sportolót tartalmazhat, de egy sportoló csak egy csapathoz tartozik.
- **1:1 kapcsolat** a Verseny és a Helyszín között, hiszen minden verseny egy adott helyszínen kerül megrendezésre.
- **M:N kapcsolat** a Csapat és a Verseny között, amelyet a Részvétel kapcsolatmodellel valósítottam meg. Ez a kapcsolat tartalmazza a helyezés, pontszám és díjazás adatait.
- **1:N kapcsolat** a Szövetség és a Csapat között, mivel egy sportági szövetség több csapatot is felügyelhet, de egy csapat csak egy szövetséghez tartozik.

A kapcsolatok irányát és kardinalitását a modellben nyilakkal jelöltem, ahol az egyszeres nyíl (1) az „egy” kapcsolatot, a dupla nyíl (N) pedig a „több” kapcsolatot mutatja.

A modell megtervezése során külön figyelmet kaptak a kulcsattribútumok (pl. sportolo_id, csapat_id, verseny_id, helyszin_id, szovetseg_id) egyértelmű megadása, valamint az összetett és többértékű attribútumok helyes megjelenítése.

Az ER diagram elkészítése a draw.io program segítségével történt, ahol a szabványos entitás-, kapcsolat- és attribútumjelöléseket alkalmaztam.

Az 1. ábra bemutatja a sportverseny-nyilvántartás ER modelljét, amely az entitások és kapcsolataik logikai szerkezetét szemlélteti.



1. ábra A sportverseny-nyilvántartás ER modellje

1.2 Az adatbázis konvertálása XDM modellre

Az ER modell alapján elkészült az XDM (XML Data Model), amely az adatbázis szerkezetét XML formátumra konvertálva jeleníti meg. Az XDM modell célja, hogy az entitások és kapcsolatok hierarchikus struktúráját olyan formában ábrázolja, amely közvetlenül felhasználható az XML dokumentum létrehozásához és az XSD séma segítségével validálható.

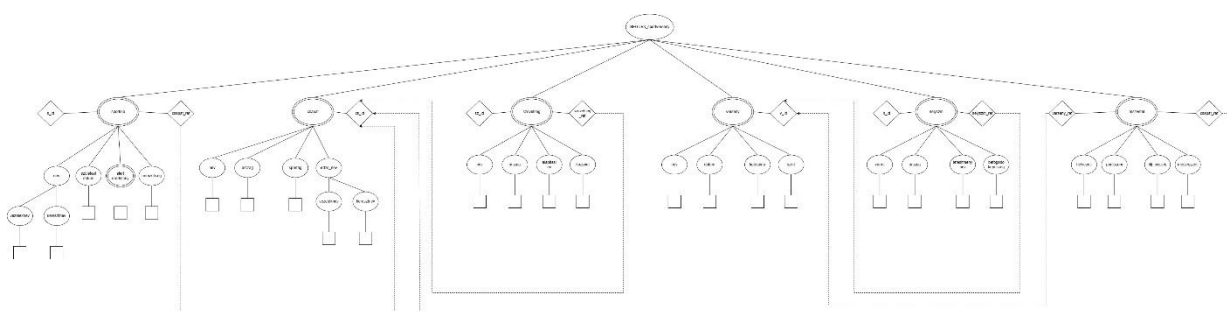
A modell gyökéreleme a Sportverseny_adatbazis, amely alá a rendszer fő entitásai szerveződnek: Sportoló, Csapat, Verseny, Helyszín, Részvétel és Szövetség. Minden entitás külön elemként jelenik meg, a hozzájuk tartozó adatok pedig al-elemekként (child node-ként) kerülnek feltüntetésre.

Az ER modellből származó kapcsolatok (1:N és M:N) az XDM-ben hierarchikus módon kerülnek leképezésre:

- A Sportoló és Csapat közötti 1:N kapcsolat úgy jelenik meg, hogy a sportoló adatai önállóan szerepelnek, és szükség esetén a csapathoz tartozó azonosítóval kapcsolhatók.
- A Csapat és Verseny közötti M:N kapcsolatot a Részvétel elem reprezentálja, amely mindkét fél azonosítóját (csapat_id, verseny_id) tartalmazza, valamint önálló tulajdonságokat is, mint a helyezés, pontszám és díj_összeg.
- A Szövetség és Csapat közötti 1:N kapcsolat az XDM-ben úgy valósul meg, hogy minden Csapat elem tartalmaz egy szovetseg_id mezőt, amely a szövetséghez való hivatkozást biztosítja. Ez a kapcsolat a modellben hierarchikus szinten is kifejezhető, mivel egy szövetség alá több csapat tartozhat.

Az XDM modell logikája biztosítja az XML dokumentumban a szerkezet érvényességét és az adatok közötti összekapcsolhatóságot. Az XDM-diagramot a draw.io program segítségével készítettem, ügyelve arra, hogy az elemeket összekötő vonalak (PK–FK) ne keresztezzék egymást, és az XML hierarchia szerkezete egyértelműen olvasható maradjon.

A 2. ábra a sportverseny-nyilvántartás adatainak XDM modelljét mutatja be, amely az XML dokumentum hierarchikus struktúráját és kapcsolatainak logikai felépítését szemlélteti.



2. ábra A sportverseny-nyilvántartás XDM modellje

1.3 Az XDM modell alapján XML dokumentum készítése

Az XDM modell alapján elkészült a SEGUV3_XML.xml nevű XML-dokumentum, amely a sportverseny-nyilvántartó rendszer adatait hierarchikus, strukturált formában tartalmazza. A dokumentum célja, hogy a rendszerben szereplő entitásokat – Sportoló, Csapat, Verseny, Helyszín, Szövetség és Résztétel – egységes, géppel feldolgozható formában reprezentálja. Ez a szerkezet megteremti a kapcsolatot a logikai adatmodell és az XML-alapú feldolgozás között.

A fájl gyökéreleme a <Sportverseny_adatbazis>, amely alá az egyes entitások csoportosítva, ismétlődő elemek formájában kerültek elhelyezésre. A hierarchia az XDM modell logikai struktúráját követi, vagyis minden fő entitás külön elemként jelenik meg, és az adott egyedhez tartozó jellemzők al-elemként (child node) szerepelnek.

A többértékű attribútumokat, például a sportolók elért_eredményeit ismétlődő al-elemek (<eredmeny>) segítségével valósítottam meg, ezáltal egy sportolóhoz több eredmény is rögzíthető.

A dokumentumban kommenteket (<!-- -->) használtam az entitások logikai szétválasztására és a példányok áttekinthetőbbé tételére. A fájl az XML Schema (XSD) alapján validálható, amely biztosítja az adatok típushelyességét, a kötelező elemek meglétét és a dokumentum szerkezeti érvényességét.

A dokumentum szerkezeti felépítése:

- Sportoló (<Sportolo>) Tartalmazza a sportoló személyes adatait: név (összetett elem: vezetéknév + keresztnév), nemzetiség, születési dátum, valamint az elért eredmények listáját.
Minden sportoló rendelkezik egy sportolo_id attribútummal és egy csapat_ref hivatkozással, amely a csapathoz való kapcsolatot jelöli.
- Csapat (<Csapat>) Egy sportági egységet képvisel, és tartalmazza a csapat nevét, sportágát, edzőjének nevét (összetett elem), valamint az országot. Minden csapathoz tartozik egy csapat_id azonosító és egy szovetseg_ref hivatkozás a szövetségre.
- Verseny (<Verseny>) A sportesemények adatait tárolja: név, dátum, kategória és szint. A helyszin_ref attribútum segítségével kapcsolódik a verseny helyszínéhez.

- Helyszín (<Helyszin>) A verseny lebonyolításának helyét írja le, várossal, országgal, létesítmény nevével és befogadóképességgel. Egyedi azonosítója a helyszin_id.
- Szövetség (<Szovetseg>) Az adott sportágért felelős hivatalos szervezetet reprezentálja. A szövetség adatai: név, sportág, alapítás éve és központ. Az szovetseg_id attribútum biztosítja az egyértelmű azonosítást.
- Résztétel (<Reszvetel>) A Csapat és a Verseny közötti kapcsolatot írja le. Attribútumai: reszvetel_id, csapat_ref, verseny_ref, valamint al-elemei: helyezés, pontszám és díj_összeg.

A dokumentum főbb szerkezeti egységei az alábbi XML-részletben láthatók:

```
<Sportolo sportolo_id="S001" csapat_ref="C001">
```

```
<nev>
```

```
<vezeteknev>Kovács</vezeteknev>
```

```
<keresztnev>Péter</keresztnev>
```

```
</nev>
```

```
<nemzetiseg>Magyar</nemzetiseg>
```

```
<szuletesi_datum>1998-04-12</szuletesi_datum>
```

```
<elert_eredmenyek>
```

```
<eredmeny>1. hely - Országos Bajnokság 2023</eredmeny>
```

```
<eredmeny>3. hely - Régiós Verseny 2022</eredmeny>
```

```
</elert_eredmenyek>
```

```
</Sportolo>
```

1.4 Az XML dokumentum alapján XMLSchema készítése

A SEGUV3_XML.xml dokumentum szerkezetének és tartalmának érvényességét a SEGUV3_XMLSchema.xsd séma biztosítja. Az XML Schema célja, hogy formálisan meghatározza az elemek és attribútumok típusát, struktúráját és kapcsolatait, valamint ellenőrizze az adatok közötti logikai integritást (kulcsok és hivatkozások révén).

A séma hierarchikus felépítésben tükrözi az XML dokumentum szerkezetét, és hat fő komplex típust definiál, amelyek megfelelnek az adatmodell entitásainak:

- SportoloTipus
- CsapatTipus
- VersenyTipus
- HelyszinTipus
- SzovetsegTipus
- ReszvetelTipus

Ezek a komplex típusok tartalmazzák az adott entitás al-elemeit és attribútumait, biztosítva a dokumentum strukturális konzisztenciáját.

A séma több saját komplex típust hoz létre (xs:complexType), amelyek az XML fő entitásainak szerkezetét írják le. Ahol szükséges volt, összetett elemeket is alkalmaztam, például a nev típust, amely vezetéknév és keresztnév al-elemekből áll.

A SportoloTipus például így épül fel:

```
<xs:complexType name="SportoloTipus">
```

```
<xs:sequence>
```

```
<xs:element name="nev" type="NevTipus"/>
```

```
<xs:element name="szuletesi_datum" type="xs:date"/>
```

```
<xs:element name="elert_eredmenyek" type="EredmenyekTipus"  
minOccurs="0"/>
```

```
</xs:sequence>
```

```
<xs:attribute name="sportolo_id" type="xs:string"  
use="required"/>
```

```
<xs:attribute name="csapat_ref" type="xs:string"
use="required"/>
```

```
</xs:complexType>
```

Ez a típus határozza meg, hogy egy sportoló milyen adatokat tartalmazhat, valamint azt, hogy a `sportolo_id` és `csapat_ref` attribútumok kötelezőek. A `EredmenyekTipus` több előfordulású (`maxOccurs="unbounded"`) eredmény elemeket enged meg, így egy sportolóhoz tetszőleges számú eredmény tartozhat.

Kapcsolatok és kulcsok definiálása

Az entitások közötti kapcsolatok kulcsok (`xs:key`) és idegen kulcsok (`xs:keyref`) segítségével kerültek megvalósításra.

Ezek az elemek garantálják, hogy az XML dokumentumon belüli hivatkozások érvényesek legyenek — például egy `Reszvetel` elemben szereplő `csapat_ref` csak akkor lehet helyes, ha az adott `Csapat` létezik.

A séma kulcsrészlete:

```
<!-- Elsődleges kulcsok -->
```

```
<xs:key name="csapat_kulcs">
```

```
<xs:selector xpath="Csapat"/>
```

```
<xs:field xpath="@csapat_id"/>
```

```
</xs:key>
```

```
<xs:key name="szovetseg_kulcs">
```

```
<xs:selector xpath="Szovetseg"/>
```

```
<xs:field xpath="@szovetseg_id"/>
```

```
</xs:key>
```

```
<!-- Idegen kulcsok -->
```

```
<xs:keyref name="csapat_ref_szovetseg" refer="szovetseg_kulcs">
```

```
<xs:selector xpath="Csapat"/>
```

```
<xs:field xpath="@szovetseg_ref"/>
```

```
</xs:keyref>
```

Ebben a példában a csapat_ref_szovetseg hivatkozás biztosítja, hogy minden csapat csak olyan szovetseg_ref értéket tartalmazhat, amely létezik a Szovetseg elemek között.

Hasonló módon, a reszvetel_ref_csapat és reszvetel_ref_versenys kulcsok érvényesítik a Részvétel – Csapat és Részvétel – Verseny kapcsolatait.

Adatintegritás és érvényesség

A séma gondoskodik:

- az adattípusok helyességéről (xs:date, xs:integer, xs:string);
- az attribútumok kötelező meglétéről (use="required");
- az ismételődő elemek korlátozásáról (maxOccurs="unbounded");
- a logikai hivatkozások konzisztenciájáról (kulcs–kulcshivatkozás mechanizmus).

Ez az XML sémát relációs adatbázisok idegen kulcsos szerkezetének megfelelően építi fel, így az adatok összefüggései szigorúan érvényesíthetők.

Megjegyzések és dokumentálás

A sémafájlban kommenteket (<!-- ... -->) használtam az egyes entitások és kapcsolatcsoportok elkülönítésére.

Ez elősegíti az átláthatóságot és a későbbi bővíthetőséget.

Minden entitás egyértelműen elkülönül, így a sémát könnyen lehet bővíteni vagy más XML-dokumentumhoz is újra felhasználni.

2. DOM program

Ebben a feladatban egy DOM alapú XML feldolgozó programot kellett készíteni, amely a SEGUV3XML.xml dokumentum adatait olvassa be és írja ki a konzolra blokkos formában. A program a DocumentBuilderFactory és DocumentBuilder osztályok segítségével beolvassa az XML-t, majd DOM fa szerkezetet hoz létre, amelyben az egyes elemeket és azok gyermekeit bejárja, és azok értékeit jeleníti meg.

2.1 Adatolvasás

Ebben a részben egy DOM alapú XML feldolgozó program készült, amely a SEGUV3XML.xml dokumentum adatait olvassa be és írja ki a konzolra blokkos formában. A program a DOM (Document Object Model) szabvány elemeit használja, így az XML dokumentum teljes szerkezete bejárható és az adatok egyenként elérhetők.

A megvalósítás célja az volt, hogy a sportverseny-nyilvántartás XML állományának adatai (Sportoló, Csapat, Verseny, Helyszín, Szövetség, Részvétel) jól strukturált, áttekinthető formában kerüljenek kiírásra a konzolra. A program a fájl teljes feldolgozását elvégzi, és minden entitás adatait külön blokkban jeleníti meg.

A program működésének fő lépései

- XML dokumentum beolvasása: A File és DocumentBuilderFactory osztályok segítségével a program megnyitja a SEGUV3XML.xml fájlt, majd a DocumentBuilder-rel létrehozza a dokumentum DOM-fáját.
- DOM fa normalizálása: A normalize() metódus gondoskodik az XML-szerkezet rendezéséről, eltávolítja az üres csomópontokat és összevonja a szövegrészeket. Ez biztosítja, hogy a feldolgozás során a program konzisztens adatstruktúrával dolgozzon.
- Adatok bejárása és kiírása blokkos formában: A getElementsByTagName() metódus segítségével a program sorra veszi az egyes fő elemeket (Sportoló, Csapat, Verseny, stb.). Minden entitás adatai külön szekcióban jelennek meg, jól tagolt és olvasható formátumban.
- Adatok mentése új XML fájlba: A program a beolvasott DOM-fát a Transformer segítségével egy új XML fájlba írja ki (SEGUV3_XML_MENTES.xml néven), demonstrálva az XML beolvasás–feldolgozás–mentés folyamatát DOM technológiával.

Lényeges kódrészlet (kommentekkel együtt):

```
// XML fájl megnyitása

File xmlFile = new
File("SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3XML.xml");

// DocumentBuilderFactory létrehozása és DocumentBuilder
példányosítása

DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();

DocumentBuilder dBuilder = factory.newDocumentBuilder();

// DOM fa létrehozása

Document doc = dBuilder.parse(xmlFile);

doc.getDocumentElement().normalize();

System.out.println("=====
==");

System.out.println(" Gyökérelem: " +
doc.getDocumentElement().getNodeName());

System.out.println("=====
==");
```

Ez a kódrészlet felelős az XML fájl beolvasásáért, a DOM-fa létrehozásáért és a szerkezet normalizálásáért.

A további lépések során a program minden fő entitást bejár, és azok adatait blokkonként, olvasható formában jeleníti meg a konzolra.

A feldolgozás végén a program egy új XML állományt is létrehoz, amely a beolvasott adatok mentett változatát tartalmazza.

Forrás (GitHub link):

https://github.com/KovacsAttilaMarcell/SEGUV3WebXML/blob/main/SEGUV3_XMLTask/SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3DOMRead.java

2.2 Adat-lekérdezés

Ebben a részben egy DOM alapú lekérdező program készült, amely az SEGUV3XML.xml dokumentumból különböző információkat jelenít meg a konzolon. A cél az volt, hogy az XML dokumentum adatait nem XPath, hanem a DOM szabvány elemeit használva kérdezzük le, vagyis a csomópontok bejárásával és azok attribútumainak, valamint gyerekelemeinek vizsgálatával.

A program a DocumentBuilderFactory és DocumentBuilder segítségével beolvassa az XML dokumentumot, létrehozza a DOM fát, majd több különböző lekérdezést hajt végre az adatszerkezeten.

A program működése

1. XML dokumentum beolvasása és DOM fa létrehozása: A DocumentBuilderFactory és DocumentBuilder segítségével a program beolvassa az XML fájlt, majd létrehozza a DOM-fát, amely a dokumentum teljes szerkezetét memóriában tárolja.
2. A DOM fa normalizálása: A normalize() metódus gondoskodik arról, hogy az XML szerkezete konzisztens legyen (azaz ne tartalmazzon üres szövegrészeket, töréseket).
3. Lekérdezések végrehajtása: A program összesen négy különböző lekérdezést hajt végre, mindegyiket a DOM-fa bejárásával, ciklusok és elemellenőrzések segítségével.

Minden lekérdezés DOM szintű bejárással valósul meg, az Element és NodeList objektumok segítségével.

A program által megvalósított 4 lekérdezés

- Magyar sportolók listázása: Ez a lekérdezés a Sportolo elemek nemzetiseg mezője alapján szűr.
- Nemzetközi versenyek megjelenítése: A program megkeresi azokat a versenyeket, amelyek szint eleme „Nemzetközi” értéket tartalmaz.
- Csapatok, amelyek 80 pontnál többet szereztek: A program a Reszvetel elemeket vizsgálja, és kiírja azokat, ahol a pontszám értéke 80 felett van. Egyúttal megjeleníti a hozzá tartozó csapat azonosítóját (csapat_ref) és helyezését is.
- Nagy befogadóképességű helyszínek listázása: A program a Helyszin elemeket elemzi, és csak azokat jeleníti meg, ahol a befogado_kepesseg értéke 10 000 fönnél nagyobb.

Lényeges kódrészlet

```
// XML fájl beolvasása

File xmlFile = new
File("SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3XML.xml");

// DOM parser előkészítése

DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();

DocumentBuilder dBuilder = factory.newDocumentBuilder();

Document doc = dBuilder.parse(xmlFile);

doc.getDocumentElement().normalize();

System.out.println("Gyökérelem: " +
doc.getDocumentElement().getNodeName());

System.out.println("=====");

// 1. Magyar sportolók listája

System.out.println("Magyar sportolók listája:");

NodeList sportolok = doc.getElementsByTagName("Sportolo");

for (int i = 0; i < sportolok.getLength(); i++) {

Element elem = (Element) sportolok.item(i);

String nemzetiseg =
elem.getElementsByTagName("nemzetiseg").item(0).getTextContent()
;

if (nemzetiseg.equalsIgnoreCase("Magyar")) {

String vezeteknev =
elem.getElementsByTagName("vezeteknev").item(0).getTextContent()
;
```



```

String keresztnev =
elem.getElementsByTagName("keresztnev").item(0).getTextContent()
;

System.out.println(" - " + vezeteknev + " " + keresztnev);

}

}
System.out.println("\n=====");
;

```

A fenti részletben a program beolvassa az XML dokumentumot, normalizálja a DOM fát, majd a `getElementsByTagName("Sportolo")` segítségével lekéri az összes Sportoló elemet. Ezután minden sportolónál lekérdezi a nemzetiséget, és ha az megegyezik a „Magyarország” értékkel, akkor a sportoló neve és nemzetisége megjelenik a konzolon.

Forrás (GitHub link):

https://github.com/KovacsAttilaMarcell/SEGUV3WebXML/blob/main/SEGUV3_XMLTask/SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3DOMQuery.java

2.3 Adatmódosítás

Ebben a részben egy DOM-alapú XML-módosító program készült, amely a SEGUUV3XML.xml dokumentum adatait módosítja, és a változásokat a konzolra is kiírja. A program a DOM API elemeit használja, tehát közvetlenül a dokumentumfa (DOM tree) csomópontjait éri el, és azok tartalmát vagy attribútumait módosítja.

A program a DocumentBuilderFactory, DocumentBuilder, valamint a Transformer osztályok segítségével:

- beolvassa az XML dokumentumot,
- elvégzi a módosításokat (például értékcsere, új elem beszúrása, attribútum változtatása),
- majd az eredményt UTF-8 kódolással kiírja a konzolra.

A programban négy különböző módosítás történt az XML dokumentum adatai alapján:

1. Új sportoló hozzáadása: A program létrehoz egy új <Sportolo> elemet, amely tartalmazza a sportoló adatait, mint a név, nemzetiség, születési dátum és elért eredmények.
2. Csapat országának módosítása: A program az első <Csapat> elemhez tartozó <ország> értékét módosítja, például „Magyarország” → „Szlovákia”.
3. Helyszín befogadóképességének növelése: A program az első <Helyszin> elem befogado_kepesseg értékét megnöveli 2000 fővel. Ehhez beolvassa a jelenlegi értéket, majd módosítja és visszaírja az új értéket az XML fába.
4. Szövetség központjának módosítása: A program az első <Szovetseg> elem <kozpont> mezőjét módosítja, például „Budapest” → „Debrecen” értékre. A módosítás szintén DOM szintű, a getElementsByTagName() és setTextContent() metódusokkal történik.

Minden módosítás után a program a konzolra írja az elvégzett változtatásokat, valamint a teljes frissített XML-struktúrát.

Lényeges kódrészlet:

```
// XML beolvasása

File inputFile = new
File("SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3XML.xml");

DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();

DocumentBuilder builder = factory.newDocumentBuilder();

Document doc = builder.parse(inputFile);

doc.getDocumentElement().normalize();

System.out.println("=====");

System.out.println(" Gyökérelem: " +
doc.getDocumentElement().getNodeName());

System.out.println("=====");

// ===== 1. Új sportoló hozzáadása =====

Element ujSportolo = doc.createElement("Sportolo");

ujSportolo.setAttribute("sportolo_id", "S003");

Element nev = doc.createElement("nev");

Element vezeteknev = doc.createElement("vezeteknev");

vezeteknev.setTextContent("Tóth");

Element keresztnév = doc.createElement("keresztnév");
```

```
keresztnev.setTextContent("László");
```

```
nev.appendChild(vezeteknev);
```

```
nev.appendChild(keresztnev);
```

```
ujSportolo.appendChild(nev);
```

```
Element nemzetiseg = doc.createElement("nemzetiseg");
```

```
nemzetiseg.setTextContent("Magyar");
```

```
ujSportolo.appendChild(nemzetiseg);
```

```
Element szuletesiDatum = doc.createElement("szuletesi_datum");
```

```
szuletesiDatum.setTextContent("2000-05-21");
```

```
ujSportolo.appendChild(szuletesiDatum);
```

```
Element eredmények = doc.createElement("elert_eredmenyek");
```

```
Element eredmény1 = doc.createElement("eredmeny");
```

```
eredmeny1.setTextContent("1. hely - Városi bajnokság 2024");
```

```
eredmenyek.appendChild(eredmeny1);
```

```
ujSportolo.appendChild(eredmenyek);
```

```
doc.getDocumentElement().appendChild(ujSportolo);
```

```
System.out.println("Új sportoló hozzáadva: Tóth László");
```

Forrás (GitHub link):

https://github.com/KovacsAttilaMarcell/SEGUV3WebXML/blob/main/SEGUV3_XMLTask/SEGUV3DOMParse/src/seguv3/domparse/hu/SEGUV3DOMModify.java

